

C.1 引言

本书的总体目标是描绘计算机如何在多个层面上工作，从构成它们的晶体管一直到它们运行的软件。本书前五章逐步讲解较低层次的抽象，从晶体管到逻辑门再到逻辑设计。第6章至第8章跃升至架构层面，并回溯至微架构，以连接硬件与软件。本附录关于C语言编程的内容在逻辑上介于第5章和第6章之间，将C语言编程作为文本中最高层次的抽象进行讲解。它既为架构内容提供动机，又将本书与读者可能已熟悉的编程经验联系起来。本材料置于附录中，以便读者可根据自身经验轻松选择学习或跳过。

程序员使用多种不同的语言来告诉计算机该做什么。从根本上说，计算机处理由1和0组成的*machine language*指令，这一点将在第6章中探讨。但用机器语言编程既繁琐又缓慢，因此程序员会使用更抽象的语言来更高效地表达其意图。表eC.1列出了一些不同抽象层次的语言示例。

有史以来最流行的编程语言之一被称为C语言。它是由包括丹尼斯·里奇和布莱恩·克尼汉在内的一组人在1969年至1973年间于贝尔实验室创建的，目的是将UNIX操作系统从最初的汇编语言重写。从许多方面来看，C语言（包括一系列紧密相关的语言，如C++、C#和Objective C）是现存使用最广泛的语言。其流行源于多个因素，包括：

在从超级计算机到嵌入式微控制器 {v*} 的众多平台上均可使用，用户群体庞大，相对易于使用

Application Software	
Operating Systems	
Architecture	
Micro-architecture	
Logic	
Digital Circuits	
Analog Circuits	
Devices	
Physics	

表 eC.1 按抽象程度大致递减的语言

Language	Description
MATLAB	Designed to facilitate heavy use of math functions
Perl	Designed for scripting
Python	Designed to emphasize code readability
Java	Designed to run securely on any computer
C	Designed for flexibility and overall system access, including device drivers
Assembly Language	Human-readable machine language
Machine Language	Binary representation of a program

中等抽象级别，比汇编语言提供更高的生产力，同时让程序员对代码的执行方式有良好的理解。

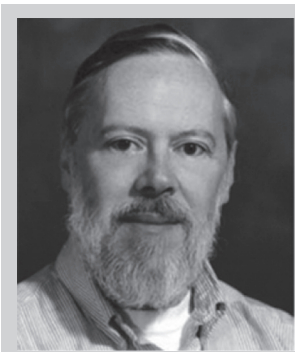
适用于生成高性能程序 ▶ 能够直接与硬件交互

本章专门介绍C语言编程，原因有多方面。最重要的是，C语言允许程序员直接访问内存地址，体现了本书强调的硬件与软件之间的联系。C语言是一种实用语言，所有工程师和计算机科学家都应掌握。它在实现与设计的诸多方面——例如软件开发、嵌入式系统编程和仿真——的应用，使得精通C语言成为一项重要且具有市场价值的技能。

以下章节描述了C程序的整体语法，讨论了程序的各个部分——包括头文件、函数和变量声明、数据类型以及库中提供的常用函数。第9章（作为网络补充材料提供——参见前言）通过使用C语言对SparkFun的RED-V RedBoard（包含RISC-V微控制器）进行编程，介绍了一个实践应用。

摘要

高级编程：高级编程在设计的多层次中都非常有用，从编写分析或仿真软件，到对与硬件交互的微控制器进行编程。▶ 低级访问：C代码之所以强大，是因为除了高级结构外，它还提供了对低级硬件和内存的访问。



丹尼斯·里奇，1941–2011



布赖恩·克尼汉（1942–）C语言于1978年由布赖恩·克尼汉和丹尼斯·里奇的经典著作

The C Programming 正式提出

Language。1989年，美国国家标准协会（ANSI）对该语言进行了扩展和标准化，形成了ANSI C、标准C或C89。此后不久，1990年，该标准被国际标准化组织（ISO）和国际电工委员会（IEC）采纳。ISO/IEC于1999年将标准更新为C99，这也是本书将要讨论的版本。

C.2 欢迎使用C语言

一个C程序是一个描述计算机要执行操作的文本文件。该文本文件被*compiled*，转换为机器可读的格式，并在计算机上运行或*executed*。C代码示例eC.1是一个简单的C程序，它将短语“Hello world!”打印到*console*（计算机屏幕）上。C程序通常包含在一个或多个以“.c”结尾的文本文件中。良好的编程风格要求文件名能指示程序的内容——例如，这个文件可以命名为hello.c。

C代码示例 eC.1 简单的C程序

```
// 向控制台输出 "Hello world!" #include <stdio.h>
int main(void) {    printf("Hello world!\n"); }
```

控制台输出

你好世界！

C . 2 . 1 C程序剖析

通常，一个C程序由一个或多个函数组成。每个程序都必须包含main函数，这是程序开始执行的地方。大多数程序还会使用在C代码其他位置和/或库中定义的其他函数。hello.c程序的整体部分包括头文件、main函数和函数体。

头文件：#include <stdio.h>

头文件包含了程序所需的*library functions*。在本例中，程序使用了printf函数，该函数是标准I/O库stdio.h的一部分。有关C语言内置库的更多详细信息，请参见C.9节。

主函数：int main(void)

所有C程序必须包含且仅包含一个main函数。程序的执行通过运行main内部的代码进行，这被称为main的*body*。函数语法在C.6节中描述。函数体包含一系列*statements*。每条语句以分号结束。int表示main函数输出或*returns*一个整数结果，该结果指示程序是否成功运行。

C是用于编写Linux、Windows和iOS等无处不在的系统的语言。由于能够直接访问硬件，C语言功能强大。与其他高级语言（例如Perl和MATLAB）相比，C语言在文件操作、模式匹配、矩阵操作和图形用户界面等专门操作方面的内置支持较少。它也缺乏保护程序员避免常见错误（例如向数组末尾之后写入数据）的特性。其强大功能与缺乏保护相结合，助长了黑客利用编写不佳的软件入侵计算机系统的行为。

本章提供了C语言编程的基础知识，但已有整本著作深入描述了C语言。我们最推崇的经典之作是C语言开发者布莱恩·克尼汉（Brian Kernighan）与丹尼斯·里奇（~~Dennis Ritchie~~^{*Dennis Ritchie*}）合著的*language*，该书简明扼要地阐述了C语言的精髓。另一本优秀教材是阿尔·凯利（Al Kelley）与艾拉·波尔（Ira Pohl）合著的*A Book on C*。

主体: `printf(“Hello world!\n”);`

该主函数体包含一条语句, 即对 `printf` 函数的调用, 该函数打印短语 “Hello world!”, 后跟由特殊序列 “\n” 表示的换行符。有关 I/O (输入/输出) 函数的更多详细信息, 请参见第 C.9.1 节。

所有程序都遵循简单 `hello.c` 程序的一般格式。当然, 非常复杂的程序可能包含数百万行代码, 并跨越数百个文件。

C.2.2 运行C程序

C程序可以在许多不同的机器上运行。这一 *portability* 是C语言的另一个优势。程序首先在目标机器上使用 *C compiler* 进行编译。存在略微不同版本的C编译器, 包括 *cc* (C编译器) 或 *gcc* (GNU C编译器)。在此, 我们展示如何使用 *gcc* 编译和运行C程序, *gcc* 可免费下载。它可直接在Linux机器上运行, 并可在Windows机器上的Cygwin环境下访问。它也适用于许多嵌入式系统, 例如SparkFun的RED-V RedBoard, 该板包含一个RISC-V微控制器。下面描述的C文件创建、编译和执行的一般过程适用于任何C程序。

1. 创建文本文件, 例如 `hello.c`。 2. 在终端窗口中, 切换到包含 `hello.c` 文件的目录, 并在命令提示符下输入 `gcc hello.c`。 3. 编译器会生成一个可执行文件。默认情况下, 该可执行文件名为 `a.out` (在Windows机器上为 `a.exe`)。 4. 在命令提示符下, 输入 `./a.out` (在Windows上为 `./a.exe`) 并按回车键。 5. 屏幕上将显示 “Hello world!”。

摘要

`filename.c`: C程序文件通常以 `.c` 扩展名命名。► `main`: 每个C程序必须且只能有一个 `main` 函数。► `#include`: 大多数C程序使用内置库提供的函数。通过在C文件顶部写入 `#include <library.h>` 来使用这些函数。► `gcc filename.c`: C文件通过编译器 (如GNU编译器 *gcc* 或C编译器 *cc*) 转换为可执行文件。► 执行: 编译后, 通过在命令行提示符下输入 `./a.out` (或 `./a.exe`) 来执行C程序。

C.3 编译

编译器是一种软件，它读取高级语言编写的程序，并将其转换为称为可执行文件的机器代码文件。关于编译器的整本教科书都有，但这里我们简要描述一下。编译器的整体操作包括：(1) 通过包含引用的库和展开宏定义来预处理文件，(2) 忽略所有不必要的信息，如注释，(3) 将高级代码转换为处理器原生、以二进制表示的简单指令，即机器语言，以及(4) 将所有指令编译成一个单一的二进制可执行文件，供计算机读取和执行。每种机器语言都针对特定的处理器，因此程序必须针对其运行的系统进行专门编译。例如，第6章将详细讨论RISC-V机器语言。

C.3.1 注释

程序员使用注释来从高层次描述代码并阐明代码功能。任何阅读过无注释代码的人都能证明其重要性。C程序使用两种类型的注释：单行注释以//开头，到行尾结束；多行注释以/*开头，以*/结束。虽然注释对于程序的组织和清晰性至关重要，但编译器会忽略它们。

```
// 这是一个单行注释的示例。 /* 这是一个多行注释的示例。 */
```

在每个C文件顶部添加注释有助于描述文件的作者、创建和修改日期以及用途。以下注释可添加在hello.c文件的顶部。

```
// hello.c // 2021年1月15日 Sarah.Harris@unlv.edu, David_Harris@hmc.edu // 该程序在屏幕上打印"Hello world!"
```

C.3.2 #define

常量使用#define 指令命名，然后在程序中通过名称使用。这些全局定义的常量也称为 *macros*。例如，假设你编写一个程序，最多允许用户猜测5次。你可以使用#define 来标识该数字。

```
#define MAXGUESSES 5
```

表示程序中的这一行将由 *pre-processor* 处理。在编译之前，预处理器会替换每个出现的位置。

C语言中的数字常量默认为十进制，但也可以是十六进制（前缀“0x”）或八进制（前缀“0”）¹。二进制常量在C99中未定义，但某些编译器支持（前缀“0b”）。例如，以下赋值是等价的：

```
char x = 37; char x = 0x25
; char x = 045; char x = 0b
100101;
```

全局定义的常量将

*magic numbers*从程序中消除。魔数是一种在程序中出现时没有名称的常量。程序中魔数的存在常常会引入棘手的错误——例如，当数字在一个位置被修改而在另一个位置未被修改时。

将程序中的标识符 MAXGUESSES 替换为 5。按照惯例，`#define` 行位于文件顶部，标识符使用全大写字母书写。通过在一个位置定义常量，然后在程序中使用该标识符，程序能够保持一致性，且该值易于修改——只需在 `#define` 行进行更改，而无需在代码中使用该值的每一行进行修改。

C 代码示例 eC.2 展示了如何使用 `#define` 指令将英寸转换为厘米。变量 `inch` 和 `cm` 被声明为 `float`，这意味着它们表示单精度浮点数。如果转换因子 (INCH2CM) 在整个大型程序中使用，通过 `#define` 声明它可以避免因拼写错误（例如，输入 2.53 而非 2.54）导致的错误，并且便于查找和修改（例如，如果需要更多有效数字）。

C代码示例 eC.2 使用#define声明常量

```
// 将英寸转换为厘米#include <stdio.h>#define INCH2C
M 2.54int main(void) { float inch = 5.5; // 5.5英寸
float cm; cm = inch * INCH2CM; printf("%f英寸 = %
f厘米\n", inch, cm);
```

```
}
```

控制台输出

```
5.500000英寸 = 13.970000厘米
```

C.3.3 #include

模块化鼓励我们将程序拆分到不同的文件和函数中。常用函数可以分组以便于复用。位于 *header file* 中的变量声明、定义值和函数定义，可以通过添加 `#include` 预处理指令被另一个文件使用。提供常用函数的 *Standard libraries* 就是通过这种方式访问的。例如，要使用标准输入/输出 (I/O) 库中定义的函数（如 `printf`），需要包含以下代码行。

```
#include <stdio.h>
```

包含文件的“.h”后缀表示它是一个头文件。虽然 `#include` 指令可以放在文件中被包含内容之前的任何位置，

¹The prefixes for hexadecimal, octal, and binary (0x, 0, and 0b) start with the number 0, not the letter O.

函数、变量或标识符如果需要，通常位于C文件的顶部。

程序员创建的头文件也可以被包含，但必须使用引号（" "）将文件名括起来，而不是尖括号（< >）。例如，一个名为myfunctions.h的用户创建的头文件将通过以下行被包含。

```
#include "myfunctions.h"
```

在编译时，尖括号中指定的文件会在系统目录中搜索。引号中指定的文件会在C文件所在的本地目录中搜索。如果用户创建的头文件位于其他目录，则必须包含该文件相对于当前目录的路径。

摘要

注释：C语言提供单行注释（//）和多行注释（/* */）。► #define NAME val：#define指令允许在整个程序中使用标识符（NAME）。在编译之前，所有NAME实例都会被替换为val。► #include：#include允许在程序中使用通用函数。对于内置库，需在代码顶部添加以下行：#include <library.h>。若要包含用户自定义的头文件，名称必须用引号括起来，并根据需要列出相对于当前目录的路径：即 #include "other/myFuncs.h"。

C.4 变量

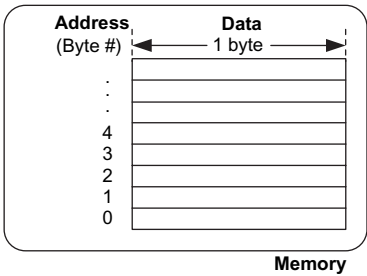
C程序中的变量具有类型、名称、值和内存位置。变量声明指明了变量的类型和名称。例如，以下声明表明该变量是char类型（一种1字节类型），且变量名为x。编译器决定将这个1字节变量放置在内存中的哪个位置。

```
char x;
```

C语言将内存视为一组连续的字节，其中每个内存字节被分配一个唯一的数字，用于指示其位置或address，如图eC.1所示。一个变量占用一个或多个内存字节；多字节变量的地址由编号最低的字节表示。变量的类型指示是否将该字节解释为整数、浮点数或其他类型。本节其余部分将描述C语言的基本数据类型、全局变量和局部变量的声明，以及变量的初始化。

变量名区分大小写，可由您自行选择。但名称不能是C语言的任何保留字（如int、while等），不能以数字开头（例如int 1x；不是有效声明），且不能包含特殊字符，如\、*、?或-。下划线（_）是允许的。

图 eC.1 C语言对内存的视角



C . 4 . 1 原始数据类型

C 语言提供了一些基本或内置的数据类型。它们大致可分为整数、浮点变量和字符。整数表示有限范围内的二进制补码或无符号数。浮点变量使用 IEEE 浮点表示法来描述具有有限范围和精度的实数。字符可视为 ASCII 值或 8 位整数。² 表 eC.2

列出了每种基本类型的大小和范围。整数可以是16位、32位或64位。除非指定为无符号，否则它们使用二进制补码表示。

表 eC.2 原始数据类型及大小

Type	Size (bits)	Minimum	Maximum
char	8	$-2^7 = -128$	$2^7 - 1 = 127$
unsigned char	8	0	$2^8 - 1 = 255$
int	machine-dependent		
unsigned int	machine-dependent		
int16_t	16	$-2^{15} = -32,768$	$2^{15} - 1 = 32,767$
uint16_t	16	0	$2^{16} - 1 = 65,535$
int32_t	32	$-2^{31} = -2,147,483,648$	$2^{31} - 1 = 2,147,483,647$
uint32_t	32	0	$2^{32} - 1 = 4,294,967,295$
int64_t	64	-2^{63}	$2^{63} - 1$
uint64_t	64	0	$2^{64} - 1$
float	32	$\pm 2^{-126}$	$\pm 2^{127}$
double	64	$\pm 2^{-1023}$	$\pm 2^{1022}$

²Technically, the C99 standard defines a character as “a bit representation that fits in a byte,” without requiring a byte to be 8 bits. However, current systems define a byte as 8 bits.

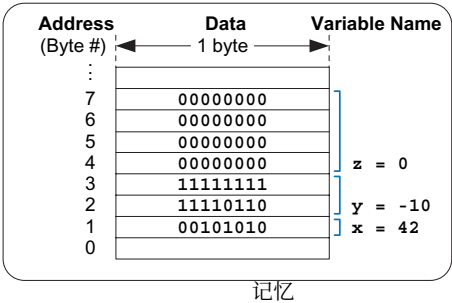


图 eC.2 C代码示例 eC.3 在内存中的变量存储

int类型的大小依赖于机器，通常是机器的原生字长。例如，在32位处理器上，int或unsigned int的大小为32位。在16位处理器上，int通常为16位。然而，64位处理器的编译器通常为int使用32位，以减少移植假设此大小的旧代码时出现的细微错误。如果你关心数据类型的大小，请使用int16_t、int32_t或int64_t来显式定义大小。（这些是有符号数据类型；它们的无符号对应类型是uint16_t等。）浮点数分别使用32位或64位表示单精度或双精度。字符为8位。

C代码示例eC.3展示了不同类型变量的声明。如图eC.2所示，x需要一个字节的数据，y需要两个字节，z需要四个字节。程序决定这些字节在内存中的存储位置，但每种类型所需的数据量始终相同。为便于说明，本例中x、y和z的地址分别为1、2和4。变量名区分大小写，因此，例如变量x和变量X是两个不同的变量。（但在同一程序中使用两者会非常令人困惑！）

C代码示例 eC.3 示例数据类型

```
// 几种数据类型及其二进制表示的示例 unsigned char x = 42; // x = 00101010 int16_t y
= -10; // y = 11111111 11110110 uint32_t z = 2; // z = 00000000 0000
000 00000000 00000010
```

C.4.2 全局变量与局部变量

全局变量和局部变量在声明位置和可见范围上有所不同。全局变量在所有函数外部声明，通常位于程序顶部，并且可以被所有函数访问。应谨慎使用全局变量，因为它们违反了模块化原则，使得大型程序更难以阅读。然而，被多个函数访问的变量可以设为全局变量。

局部变量在函数内部声明，且只能被该函数使用。因此，两个函数可以拥有同名的局部变量。

变量的scope是指它可被使用的上下文。例如，对于局部变量而言，其作用域是声明它的函数。在其他任何地方，它都不在作用域内。

同名变量互不干扰。局部变量在函数开头声明，函数结束时消亡，再次调用时重新创建。它们不会在函数的一次调用与下一次调用之间保留其值。

C代码示例eC.4和eC.5比较了使用全局变量与局部变量的程序。在C代码示例eC.4中，全局变量max可以被任何函数访问。而使用局部变量，如C代码示例eC.5所示，是更优的风格，因为它保持了模块化良好定义的接口。

C代码示例 eC.4 全局变量

```
// Use a global variable to find and print the maximum of 3 numbers
int max;           // global variable holding the maximum value
void findMax(int a, int b, int c) {
    max = a;
    if (b > max) {
        if (c > b) max = c;
        else     max = b;
    } else if (c > max) max = c;
}
void printMax(void) {
    printf("The maximum number is: %d\n", max);
}
int main(void) {
    findMax(4, 3, 7);
    printMax();
}
```

C代码示例 eC.5 局部变量

```
// Use local variables to find and print the maximum of 3 numbers
int getMax(int a, int b, int c) {
    int result = a; // local variable holding the maximum value
    if (b > result) {
        if (c > b) result = c;
        else     result = b;
    } else if (c > result) result = c;
    return result;
}
void printMax(int m) {
    printf("The maximum number is: %d\n", m);
}
int main(void) {
    int max;
    max = getMax(4, 3, 7);
    printMax(max);
}
```

C.4.3 初始化变量

变量在被读取之前，必须先被赋值——即为其指定一个值。当声明一个变量时，系统会在内存中为该变量保留正确的字节数。然而，这些内存位置会保留上次使用时的任意值，本质上是一个随机值。全局变量和局部变量可以在声明时初始化，也可以在程序主体内初始化。C代码示例eC.3展示了变量在声明的同时进行初始化。C代码示例eC.4展示了变量在声明之后、使用之前如何被初始化；全局变量max由getMax函数初始化，随后被printMax函数读取。读取未初始化的变量是一种常见的编程错误，且可能难以调试。

摘要

变量：每个变量由其数据类型、名称和内存位置定义。变量声明为数据类型名称。► 数据类型：数据类型描述了变量的大小（字节数）和表示方式（字节的解释）。表 eC.2 列出了 C 语言的内置数据类型。► 内存：C 语言将内存视为一个字节列表。内存存储变量，并将每个变量与一个地址（字节编号）关联。► 全局变量：全局变量在所有函数外部声明，可在程序中的任何位置访问。► 局部变量：局部变量在函数内部声明，只能在该函数内部访问。► 变量初始化：每个变量在读取前必须初始化。初始化可以在声明时或之后进行。

C.5 运算符

C程序中最常见的语句类型是*expression*，例如

```
y = a + 3;
```

表达式涉及运算符（如+或*）对一个或多个操作数（如变量或常量）进行操作。C语言支持表eC.3中所示的运算符，按类别列出，并按优先级从高到低排列。例如，乘法运算符的优先级高于加法运算符。在同一类别中，运算符按照它们在程序中出现的顺序进行求值。

表 eC.3 按优先级递减顺序列出的运算符

Category	Operator	Description	Example
Unary	++	post-increment	a++; // a = a+1
	--	post-decrement	x--; // x = x-1
	&	memory address of a variable	x = &y; // x = the memory // address of y
	~	bitwise NOT	z = ~a;
	!	Boolean NOT	!x
	-	negation	y = -a;
	++	pre-increment	++a; // a = a + 1
	--	pre-decrement	--x; // x = x - 1
	(type)	casts a variable to (type)	x = (int)c; // cast c to an int and // assign it to x
	sizeof()	size of a variable or type in bytes	int32_t y; x = sizeof(y); // x = 4
Multiplicative	*	multiplication	y = x * 12;
	/	division	z = 9 / 3; // z = 3
	%	modulo	z = 5 % 2; // z = 1
Additive	+	addition	y = a + 2;
	-	subtraction	y = a - 2;
Bitwise Shift	<<	bitshift left	z = 5 << 2; // z = 0b00010100
	>>	bitshift right	x = 9 >> 3; // x = 0b00000001
Relational	==	equals	y == 2
	!=	not equals	x != 7
	<	less than	y < 12
	>	greater than	val > max
	<=	less than or equal	z <= 2
	>=	greater than or equal	y >= 10

表 eC.3 按优先级递减顺序列出的运算符

—续

Category	Operator	Description	Example
Bitwise	&	bitwise AND	y = a & 15;
	^	bitwise XOR	y = 2 ^ 3;
		bitwise OR	y = a b;
Logical	&&	Boolean AND	x && y
		Boolean OR	x y
Ternary	? :	ternary operator	y = x ? a : b; // if x is TRUE, // y = a, else y = b
Assignment	=	assignment	x = 22;
	+=	addition and assignment	y += 3; // y = y + 3
	-=	subtraction and assignment	z -= 10; // z = z - 10
	*=	multiplication and assignment	x *= 4; // x = x * 4
	/=	division and assignment	y /= 10; // y = y / 10
	%=	modulo and assignment	x %= 4; // x = x % 4
	>>=	bitwise right-shift and assignment	x >>= 5; // x = x >> 5
	<<=	bitwise left-shift and assignment	x <<= 2; // x = x << 2
	&=	bitwise AND and assignment	y &= 15; // y = y & 15
	=	bitwise OR and assignment	x = y; // x = x y
	^=	bitwise XOR and assignment	x ^= y; // x = x ^ y

一元运算符，也称为单目运算符，只有一个操作数。三元运算符有三个操作数，而其他所有运算符都有两个操作数。三元运算符（源自拉丁语 *ternarius*，意为由三部分组成）根据第一个值是 TRUE（非零）还是 FALSE（零），分别选择第二个或第三个操作数。C 代码示例 eC.6 展示了如何使用三元运算符计算 `y = max(a,b)`，以及一个等效但更冗长的 `if/else` 语句。

真相，全部真相，唯有真相

C语言将非零变量视为TRUE，将零变量视为FALSE。逻辑运算符、三元运算符以及诸如if和while之类的控制流语句都依赖于变量的真值。关系运算符和逻辑运算符在结果为TRUE时返回1，在结果为FALSE时返回0。

C Code Example eC.6 (a) TERNARY OPERATOR, AND (b) EQUIVALENT if/else STATEMENT

```
(a) y = (a > b) ? a : b; // parentheses not necessary, but makes it clearer
(b) if (a > b) y = a;
    else y = b;
```

简单赋值使用=运算符。C语言还允许复合赋值，即在简单运算（如加法+=或乘法*=）之后进行赋值。在复合赋值中，左侧的变量既参与运算，又被赋予运算结果。C代码示例eC.7展示了这些操作及其他C操作。注释中的二进制值以前缀“0b”表示。

C代码示例 eC.7 运算符示例

Expression	Result	Notes
53 / 14	3	Integer division truncates
53 % 14	11	53 mod 14
0x2C && 0xE // 0b101100 && 0b1110	1	Logical AND
0x2C 0xE // 0b101100 0b1110	1	Logical OR
0x2C & 0xE // 0b101100 & 0b1110	0xC (0b001100)	Bitwise AND
0x2C 0xE // 0b101100 0b1110	0x2E (0b101110)	Bitwise OR
0x2C ^ 0xE // 0b101100 ^ 0b1110	0x22 (0b100010)	Bitwise XOR
0xE << 2 // 0b1110 << 2	0x38 (0b111000)	Left shift by 2
0x2C >> 3 // 0b101100 >> 3	0x5 (0b101)	Right shift by 3
x = 14; x += 2;	x = 16	
y = 0x2C; // y = 0b101100 y &= 0xF; // y = y & 0b1111	y = 0xC (0b001100)	
x = 14; y = 83; y = y + x++;	x = 15, y = 97	Increment x after using it
x = 14; y = 83; y = y + ++x;	x = 15, y = 98	Increment x before using it

C.6 函数调用

模块化是良好编程的关键。大型程序被划分为称为函数的较小部分，类似于硬件模块，这些函数具有明确定义的输入、输出和行为。C代码示例eC.8展示了sum3函数。函数声明以返回类型int开头，接着是名称sum3，以及括号内的输入(int a, int b, int c)。花括号{}包含函数体，其中可以包含零个或多个语句。return语句指示函数应返回给调用者的值；这可以视为函数的输出。一个函数只能返回单个值。

函数也被称为过程。

C代码示例 eC.8 sum3 函数

```
// 返回三个输入变量的和int sum3(int a, int b, int c) { int res
ult = a + b + c; return result;

}
```

在调用 sum3 之后，y 持有值 42。

```
int y = sum3(10, 15, 17);
```

尽管函数可能有输入和输出，但两者都不是必需的。C代码示例eC.9展示了一个没有输入或输出的函数。函数名前的关键字void表示不返回任何内容。括号中的void表示该函数没有输入参数。

C代码示例 eC.9 函数 printPrompt 无输入或输出

```
// 在控制台打印提示信息void printPrompt(void) { printf("请
输入一个从1-3的数字: \n");}
```

括号内没有任何内容也表示没有输入参数。因此，在这种情况下，我们可以写成：

```
void printPrompt()
```

函数在调用前必须在代码中声明。这可以通过将被调用函数放在文件中的更早位置来实现。因此，main函数通常放在C文件的末尾，位于它所调用的所有函数之后。或者，可以在函数定义之前将函数prototype放在程序中。函数原型是第一行

通过精心安排函数的顺序，原型可能并非必需。然而，在某些情况下，它们不可避免，例如当函数 `f1` 调用 `f2` 且 `f2` 调用 `f1` 时。良好的编程风格是将程序中所有函数的原型放在 C 文件的开头附近或头文件中。

与变量名一样，函数名区分大小写，不能是C语言的任何保留字，不能包含特殊字符（下划线除外），且不能以数字开头。通常，函数名包含一个动词以表明其功能。

保持函数和变量名称的大小写风格一致，这样就不必反复确认正确的大小写。两种常见风格是驼峰式命名（`camelCase`），即除首个单词外每个单词首字母大写，形似骆驼驼峰（例如 `printPrompt`）；或使用下划线连接单词（例如 `print_prompt`）。我们非科学地观察到，频繁按动下划线键会加剧腕管综合征（光是想到下划线，我的小指就隐隐作痛！），因此更偏好驼峰式命名。但最重要的是，在你的组织内部保持风格统一。

函数的原型；它声明了返回类型、函数名称和函数输入。例如，C代码示例eC.8和eC.9中函数的函数原型为：

```
int sum3(int a, int b, int c); void printPrompt(void);
```

C代码示例eC.10展示了函数原型的使用方式。尽管函数本身位于`main`之后，但文件顶部的函数原型允许它们在`main`中被调用。

C代码示例 eC.10 函数原型

```
#include <stdio.h> // 函数原型
int sum3(int a, int b, int c);
void printPrompt(void);
int main(void) {
    int y = sum3(10, 15, 20);
    printf("sum3 结果: %d\n", y);
    printPrompt();
}
int sum3(int a, int b, int c) {
    int result = a + b + c;
    return result;
}
void printPrompt(void) {
    printf("请输入一个从1到-3的数字:\n");
}
```

控制台输出

sum3 结果: 45 请输入一个数字 1-3:

`main` 函数始终声明返回一个 `int` 类型；该返回值告知操作系统程序终止的原因。零表示正常结束，而非零值则指示错误状态。如果 `main` 函数执行到末尾而未遇到 `return` 语句，它将自动返回 `0`。大多数操作系统不会自动告知用户程序返回的值。

C.7 控制流语句

C 提供了用于条件判断和循环的控制流语句。条件判断仅在满足条件时执行语句。循环则在满足条件时重复执行语句。

C.7.1 条件语句

在高级语言（包括C语言）中，if、if/else 和 switch/case 语句是常用的条件语句。

条件语句

当括号中的表达式为真（即非零）时，if 语句会执行紧随其后的语句。其一般格式为：

if (表达式) 语句

C 代码示例 eC.11 展示了如何在 C 语言中使用 if 语句。当变量 aintBroke 等于 1 时，变量 dontFix 被设置为 1。可以通过在语句周围放置花括号 {} 来执行多个语句的块，如 C 代码示例 eC.12 所示。

花括号{}用于将零个或多个语句分组为*compound statement*或*block*。

C代码示例 eC.11 IF语句

```
int dontFix = 0; if (aintBroke == 1) dontFix = 1;
```

C代码示例 eC.12 带语句块的IF语句

```
// 如果金额 >= $2, 提示用户并分发糖果 if (amt >= 2) { p
rintf("选择糖果.\n"); dispenseCandy = 1; }
```

If/else 语句

if/else语句根据条件执行两个语句之一，如下所示。当if语句中的表达式为TRUE时，执行statement1。否则，执行statement2。

如果 (表达式) 语句1 否则 语句2

C 代码示例 eC.6(b) 给出了一个 C 语言中的 if/else 语句示例。该代码将 y 设置为 a（如果 a 大于 b）；否则 y = b。

Switch/case 语句

Switch/case 语句根据条件执行若干语句之一，如下通用格式所示。

```
switch (variable) { case (expression1): statement1 break; case (expression2): statement2 break; case (expression3): statement3 break; default: statement4 }
```

例如，如果变量等于表达式2，则执行将继续到语句2，直到遇到关键字break，此时退出switch/case语句。如果没有满足任何条件，则执行default。

如果省略关键字 break，执行将从条件为 TRUE 的点开始，然后继续向下执行其后的所有 case 分支。这通常不是你想要的结果，也是 C 语言初学者常犯的错误。

C 代码示例 eC.13 展示了一个 switch/case 语句，该语句根据变量 option 确定要支付的金额 amt。switch/case 语句等价于一系列嵌套的 if/else 语句，如 C 代码示例 eC.14 中的等效代码所示。

C代码示例 eC.13 SWITCH/CASE语句

```
// 根据选项的值分配 amt
switch (option) { case 1: amt = 100; break; case 2: amt = 50; break; case 3: amt = 20; break; case 4: amt = 10; break; default: printf("错误: 未知选项.\n"); }

}
```

C代码示例 eC.14 嵌套IF/ELSE语句

```
// 根据选项的值分配 amt
if (option == 1) amt = 100; else if (option == 2) amt = 50; else if (option == 3) amt = 20; else if (option == 4) amt = 10; else printf("错误: 未知选项.\n");
```

C.7.2 循环

在C语言等许多高级语言中，while、do/while 和 for 循环是常见的循环结构。只要条件满足，这些循环就会重复执行一条语句。

While Loops

While循环重复执行一条语句，直到条件不满足为止，如下通用格式所示。

while (条件) 语句

C语言中的while循环示例eC.15计算了9的阶乘 $= 9 \times 8 \times 7 \times \cdots \times 1$ 。注意，条件是在执行语句之前检查的。在此示例中，语句是一个复合语句或代码块，因此需要使用花括号。

C代码示例 eC.15 WHILE循环

```
// 计算 9! (9 的阶乘)  int i = 1, fact = 1; // 将 1 到 9 的数字相乘  while (i < 10) { // while 循环先检查条件  fact *= i;  i++; }
```

Do/while 循环

Do/while 循环与 while 循环类似，但条件仅在语句执行一次后才进行检查。其一般格式如下所示。条件后面需跟一个分号。

do 语句 while (条件);

C语言中的do/while循环在代码示例eC.16中用于询问用户猜测一个数字。程序仅在do/while循环体执行一次后检查条件（即用户输入的数字是否等于正确数字）。这种结构在需要先执行某些操作（例如获取用户的猜测）再检查条件时非常有用。scanf函数（详见C.9.1节）将用户按下的键值存入变量guess中。

C代码示例 eC.16 DO/WHILE循环

```
// Query user to guess a number and check it against the correct number.
#define MAXGUESSES 3
#define CORRECTNUM 7
int guess, numGuesses = 0;
do {
    printf("Guess a number between 0 and 9. You have %d more guesses.\n",
           (MAXGUESSES-numGuesses));
    scanf("%d", &guess);    // Read user input
    numGuesses++;
} while ( (numGuesses < MAXGUESSES) & (guess != CORRECTNUM) );
// do loop checks the condition after the first iteration
if (guess == CORRECTNUM)
    printf("You guessed the correct number!\n");
```

For Loops

for循环与while和do/while循环类似，会重复执行一条语句直到条件不满足。不过，for循环额外支持一个*loop variable*，通常用于记录循环执行的次数。for循环的一般格式为

for (初始化; 条件; 循环操作) 语句

初始化代码仅在for循环开始前执行一次。条件在每次循环迭代开始时进行测试。如果条件不为真，则循环退出。循环操作在每次迭代结束时执行。C代码示例eC.17展示了使用for循环计算9的阶乘。

C代码示例 eC.17 FOR循环

```
// 计算 9! int i; // 循环变量 int
fact = 1; for (i = 1; i < 10; i++)
fact *= i;
```

在C代码示例eC.15和eC.16中，while和do/while循环包含了递增和检查循环变量*i*及numGuesses的代码，而for循环则将那些语句整合到了其格式中。for循环可以等价地、但不太方便地表示为

初始化; 当 (条件) 时 {
语句 循环操作; }

摘要

控制流语句：C语言提供了用于条件语句和循环的控制流语句。► 条件语句：条件语句在条件为真时执行一条语句。C语言包含以下条件语句：if、if/else 和 switch/case。► 循环：循环重复执行一条语句，直到条件为假。C语言提供了 while、do/while 和 for 循环。

C.8 更多数据类型

除了各种大小的整数和浮点数，C语言还包含其他特殊数据类型，包括指针、数组、字符串和结构体。本节将介绍这些数据类型以及动态内存分配。

C.8.1 指针

指针是变量的地址。C代码示例eC.18展示了如何使用指针。salary1和salary2是可以包含整数的变量，而ptr是一个可以存放整数地址的变量。编译器会根据运行时环境为这些变量在RAM中分配任意位置。具体来说，假设该程序在32位系统上编译，salary1位于地址0x70–73，salary2位于地址0x74–77，ptr位于地址0x78–7B。图eC.3显示了程序执行后的内存及其内容。

在变量声明中，变量名前的星号（*）表示该变量是指向声明类型的指针。在使用指针变量时，*运算符dereferences解引用指针，返回存储在该地址的值。

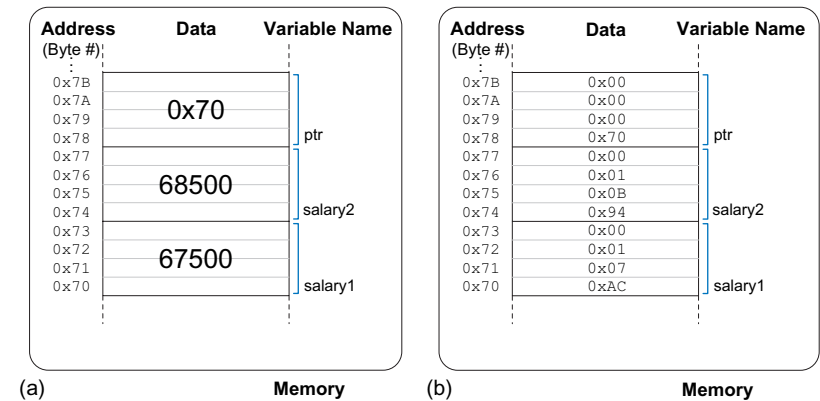


图 eC.3 内存内容 在 C 代码示例 eC.18 执行后 按值显示 (a) 和 按字节使用小端序显示 (b)

指针中包含的指示内存地址。&运算符读作“取地址”，它生成被引用变量的内存地址。

C Code Example eC.18 POINTERS

```
// Example pointer manipulations
int salary1, salary2; // 32-bit numbers
int *ptr;             // a pointer specifying the address of an int variable

salary1 = 67500;       // salary1 = $67,500 = 0x000107AC
ptr = &salary1;        // ptr = 0x0070, the address of salary1
salary2 = *ptr + 1000; /* dereference ptr to give the contents of address 70 = $67,500,
                       then add $1,000 and set salary2 to $68,500 */
```

解引用指向不存在内存位置或程序可访问范围之外地址的指针通常会导致程序崩溃。这种崩溃通常被称为segmentation fault

指针在函数需要修改变量而不仅仅是返回值时特别有用。由于函数无法直接修改其输入，因此函数可以将输入设为指向该变量的指针。这被称为传递输入变量*by reference*而非*by value*，如之前的示例所示。C代码示例eC.19给出了一个通过引用传递x的实例，使得quadruple能够直接修改该变量。

C Code Example eC.19 PASSING AN INPUT VARIABLE BY REFERENCE

```
// Quadruple the value pointed to by a
#include <stdio.h>

void quadruple(int *a) {
    *a = *a * 4;
}

int main(void) {
    int x = 5;
    printf("x before: %d\n", x);
    quadruple(&x);
    printf("x after: %d\n", x);
    return 0;
}
```

Console Output

x before: 5
x after: 20

指向地址0的指针称为*null pointer*，表示该指针实际上并未指向有意义的数据。在程序中写作NULL。

C.8.2 数组

数组是一组存储在内存连续地址中的相似变量。元素从0编号到N-1，其中N是数组的长度。C代码示例eC.20声明了一个名为scores的数组变量，用于存储三名学生的期末考试成绩。内存空间为三个32位整数预留，即3×4=12字节。假设scores数组从地址0x40开始。第0th个元素（即scores[0]）的地址是0x40，第一个元素是0x44，第二个元素是0x48，如图eC.4所示。在C语言中，数组变量（此处为scores）是指向第0th个元素的指针；换句话说，scores包含第0th个数组元素的地址。程序员有责任不访问数组末尾之后的元素。C语言没有内部边界检查，因此写入数组末尾之后的程序可以正常编译，但在运行时可能会覆盖内存的其他部分。

通俗地说，数组的“第0个”元素也被称为“第一个”数组元素。

C代码示例 eC.20 数组声明

```
int32_t scores[3]; // 三个4字节数字的数组
```

数组的元素可以在声明时使用花括号{}进行初始化，如C代码示例eC.21所示，也可以在代码主体中逐个初始化，如C代码示例eC.22所示。数组的每个元素通过方括号[]进行访问。包含数组的内存内容如图eC.4所示。使用花括号{}进行数组初始化只能在声明时进行，之后无法再进行。for循环常用于赋值和读取数组数据，如C代码示例eC.23所示。

```
int32_t scores[3] = {93, 81, 97}; // scores[0] = 93; scores[1] = 81; scores[2] = 97; C代码示例 eC.21 声明时使用{}进行数组初始化
```

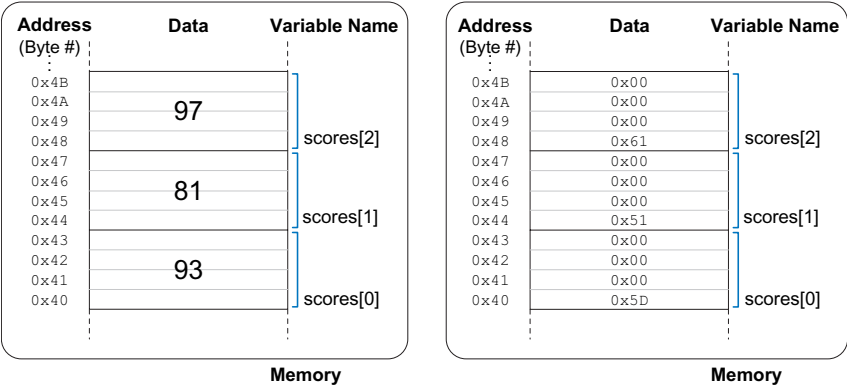


图 eC.4 存储在内存中的 scores 数组

C代码示例 eC.22 使用赋值进行数组初始化

```
int32_t scores[3]; scores[0] = 93; scores[1] = 81; scores[2] = 97;
```

C代码示例 eC.23 使用for循环进行数组初始化

```
// 用户将3个学生成绩输入到一个数组中int32_t scores[3];int i, entered;printf("请输入学生的3个成绩。\\n");for (i = 0; i < 3; i++) { printf("输入一个成绩并按回车。\\n"); scanf("%d", &entered); scores[i] = entered;}printf("成绩: %d %d %d\\n", scores[0], scores[1], scores[2]);
```

声明数组时，长度必须为常量，以便编译器能够分配适当大小的内存。然而，当数组作为输入参数传递给函数时，无需定义长度，因为函数只需知道数组起始地址。C代码示例eC.24展示了如何将数组传递给函数。输入参数arr仅表示数组第0th个元素的地址。通常，数组中的元素数量也会作为输入参数传递。在函数中，类型为int[]的输入参数表明它是一个整数数组。任何类型的数组都可以传递给函数。

C代码示例 eC.24 将数组作为输入参数传递

UMENT

```
// Initialize a 5-element array, compute the mean, and print the result.
#include <stdio.h>

// Returns the mean value of an array (arr) of length len
float getMean(int arr[], int len) {
    int i;
    float mean, total = 0;
    for (i=0; i < len; i++)
        total += arr[i];
    mean = total / len;
    return mean;
}

int main(void) {
    int data[4] = {78, 14, 99, 27};
```



```
float avg;
avg = getMean(data, 4);
printf("The average value is: %f.\n", avg);
}
```

Console Output

The average value is: 54.500000.

数组参数等价于指向数组起始位置的指针。因此，`getMean` 也可以声明为

```
float getMean(int *arr, int len);
```

尽管功能上等价，但 `datatype[]` 是传递数组作为输入参数的首选方法，因为它更清晰地表明该参数是一个数组。

函数仅限于单个输出，即返回变量。然而，通过接收数组作为输入参数，函数可以通过改变数组本身来输出多个值。C代码示例eC.25将数组从低到高排序，并将结果保留在同一数组中。以下三个函数原型是等价的。函数声明中数组的长度（例如 `int vals[100]`）会被忽略。

```
void sort(int *vals, int len); void sort(int vals[],
int len); void sort(int vals[100], int len);
```

C代码示例 eC.25 将数组及其长度作为输入传递

```
// Sort the elements of the array vals of length len from lowest to highest
void sort(int vals[], int len) {
    int i, j, temp;
    for (i = 0; i < len; i++) {
        for (j = i + 1; j < len; j++) {
            if (vals[i] > vals[j]) {
                temp = vals[i];
                vals[i] = vals[j];
                vals[j] = temp;
            }
        }
    }
}
```

数组可以具有多个维度。C代码示例eC.26使用一个二维数组来存储十名学生在八套问题集中的成绩。请注意，使用`{}`初始化数组值仅在声明时允许。

C代码示例 eC.26 二维数组初始化

```
// Initialize 2D array at declaration
int grades[10][8] = { {100, 107, 99, 101, 100, 104, 109, 117},
                      {103, 101, 94, 101, 102, 106, 105, 110},
                      {101, 102, 92, 101, 100, 107, 109, 110},
                      {114, 106, 95, 101, 100, 102, 102, 100},
                      {98, 105, 97, 101, 103, 104, 109, 109},
                      {105, 103, 99, 101, 105, 104, 101, 105},
                      {103, 101, 100, 101, 108, 105, 109, 100},
                      {100, 102, 102, 101, 102, 101, 105, 102},
                      {102, 106, 110, 101, 100, 102, 120, 103},
                      {99, 107, 98, 101, 109, 104, 110, 108} };
```

C代码示例eC.27展示了一些对C代码示例eC.26中的二维grades数组进行操作的函数。用作函数输入参数的多维数组必须定义除第一维之外的所有维度。因此，以下两个函数原型是可接受的：

```
void print2dArray(int arr[10][8]); void print2dArr
ay(int arr[][8]);
```

C代码示例 eC.27 操作多维数组

```
#include <stdio.h>

// Print the contents of a 10 × 8 array
void print2dArray(int arr[10][8]) {
    int i, j;
    for (i = 0; i < 10; i++) {        // for each of the 10 students
        printf("Row %d\n", i);
        for (j = 0; j < 8; j++) {
            printf("%d ", arr[i][j]); // print scores for all 8 problem sets
        }
        printf("\n");
    }
}

// Calculate the mean score of a 10 × 8 array
float getMean(int arr[10][8]) {
    int i, j;
    float mean, total = 0;
    // get the mean value across a 2D array
    for (i = 0; i < 10; i++) {
        for (j = 0; j < 8; j++) {
            total += arr[i][j];        // sum array values
        }
    }
    mean = total / (10 * 8);
    printf("Mean is: %f\n", mean);
    return mean;
}
```

注意，由于数组是通过指向首元素的指针来表示的，C语言无法使用==或==运算符来复制或比较数组。相反，必须使用循环逐个复制或比较每个元素。

C.8.3 字符

字符（char）是一个8位变量。它可以被视为介于-128到127之间的二进制补码数，或作为字母、数字或符号的ASCII码。ASCII字符可以指定为数值（十进制、十六进制等），或作为用单引号括起来的可打印字符。例如，字母A的ASCII码是0x41，B是=0x42，依此类推。因此“A”+3是0x44，即“D”。表6.2列出了ASCII字符编码，表eC.4列出了用于表示格式或特殊字符的字符。格式代码包括回车符（\r）、换行符（\n）、水平制表符（\t）和字符串结束符（\0）。\r仅为完整性而列出，在C程序中很少使用。 \r将打字位置返回到行首（左侧），但该行原有的任何文本都会被覆盖。而\n则将打字位置移动到新行的开头。³字符（'\0'）表示文本字符串的结束，将在下一节C.8.4中讨论。

C.8.4 字符串

字符串是一个字符数组，用于存储一段长度有限但可变的文本。每个字符是一个字节，代表ASCII码。

表 eC.4 特殊字符

Special Character	Hexadecimal Encoding	Description
\r	0x0D	carriage return
\n	0x0A	new line
\t	0x09	tab
\0	0x00	terminates a string
\\	0x5C	backslash
\"	0x22	double quote
\'	0x27	single quote
\a	0x07	bell

³Windows 文本文件使用 \r\n 表示行尾，而基于 UNIX 的系统使用 \n，这在系统间移动文本文件时可能导致严重错误。

“回车”一词源于打字机，打字机需要承载纸张的滑架向右移动，以便从页面左侧开始打字。按下下图左侧所示的回车杆，滑架会向右移动，同时纸张前进一步，这称为换行。



温斯顿·丘吉尔使用过的一台雷明顿电动打字机。 <http://cwr.iwm.org.uk/server/show/conMediaFile.71979>

C 字符串被称为 *null* 或 *zero terminated*
terminated
因为长度是通过查找末尾的零来确定的。相比之下，Pascal 等语言使用第一个字节来指定字符串长度，最大可达255个字符。这个字节被称为*prefix byte*，此类字符串被称为*P-strings*。以空字符结尾的字符串的一个优点是长度可以任意大。P-字符串的一个优点是无需检查字符串中的所有字符即可立即确定长度。

对于该字母、数字或符号。数组的大小决定了字符串的最大长度，但字符串的实际长度可能更短。在C语言中，字符串的长度通过查找字符串末尾的空终止符（ASCII值0x00）来确定。

C 代码示例 eC.28 展示了一个名为 `greeting` 的 10 元素字符数组的声明，该数组存储字符串 "Hello!"。为具体起见，假设 `greeting` 从内存地址 0x50 开始。图 eC.5 显示了从 0x50 到 0x59 的内存内容，其中存储了字符串 "Hello!"。请注意，尽管在内存中分配了十个元素，但该字符串仅使用了数组的前七个元素。

C代码示例 eC.28 字符串声明

```
char greeting[10] = "Hello!";
```

C 代码示例 eC.29 展示了字符串 `greeting` 的另一种声明方式。指针 `greeting` 保存了由 "Hello!" 中每个字符组成的 7 元素数组的第一个元素的地址。

后跟空终止符。该代码还演示了如何使用 `%s`（字符串）格式代码打印字符串。

C代码示例 eC.29 替代字符串声明

```
char *greeting = "Hello!";
printf("greeting: %s", greeting);
```

Console Output
greeting: Hello!

与基本变量不同，字符串不能使用等号运算符 `=` 将一个字符串赋给另一个字符串。字符数组中的每个元素必须从源字符串逐个复制到目标字符串。这对于任何数组都是如此。C代码示例 eC.30 将一个字符串 `src` 复制到另一个字符串 `dst`。不需要数组的大小，因为 `src` 字符串的结尾由空终止符指示。然而，`dst` 必须足够大，以免覆盖其他数据。`strcpy` 和其他字符串操作函数可在 C 的内置库中使用（参见第 C.9.4 节）。

C代码示例 eC.30 复制字符串

```
// Copy the source string, src, to the destination string, dst
void strcpy(char *dst, char *src) {
    int i = 0;

    do {
        dst[i] = src[i];    // copy characters one byte at a time
    } while (src[i++]);    // until the null terminator is found
}
```

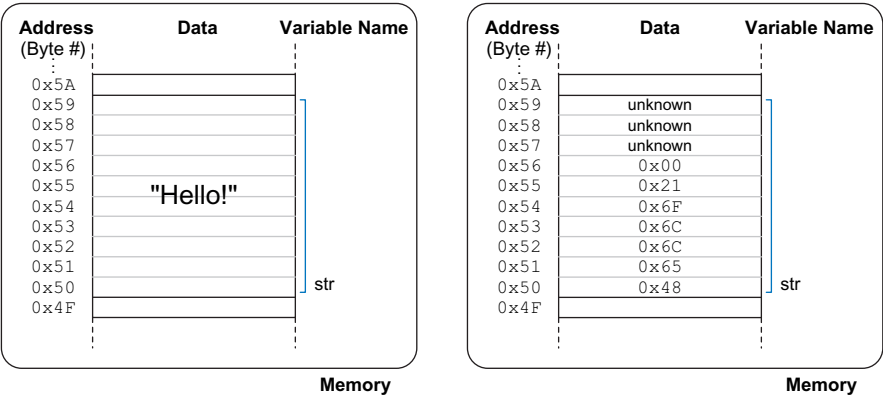


图 eC.5 字符串 "Hello!" 存储在内存中

C.8.5 结构体

在C语言中，结构体用于存储各种类型的数据集合。结构体声明的一般格式为

```
结构体名称 { 类型1
元素1; 类型2 元素2;
... };
```

struct 是一个关键字，表示它是一个结构体；name 是结构体的标签名；element1 和 element2 是结构体的成员。一个结构体可以有任意数量的成员。C 代码示例 eC.31 展示了如何使用结构体来存储联系人信息。该程序随后声明了一个类型为 struct contact 的变量 c1。

C代码示例 eC.31 结构声明

```
struct contact { char name[30]; int phone;
float height; // 以米为单位 };struct contact c1; s
trcpy(c1.name, "Ben Bitdiddle"); c1.phone = 72
26993; c1.height = 1.82;
```

与内置的C类型一样，您可以创建结构体数组和结构体指针。C代码示例 eC.32创建了一个联系人数组。

C代码示例 eC.32 结构体数组

```
结构体联系人列表[200]; 列表[0].电话 = 9642025;
```

使用指向结构的指针是很常见的。C语言提供了*member access operator* `->`来解引用指向结构的指针并访问结构的成员。C代码示例eC.33展示了一个示例，声明一个指向struct contact的指针，将其指向C代码示例eC.32中classlist的第42nd个元素，并使用成员访问运算符在该元素中设置一个值。

C代码示例 eC.33 使用指针和 -> 访问结构体成员

```
struct contact *cptr; cptr = &classlist[42]; cptr->height = 1.9; // 等价于: (*cptr).height = 1.9;
```

结构可以作为函数输入或输出，通过值传递或引用传递。按值传递要求编译器将整个结构复制到内存中供函数访问，对于大型结构而言，这可能需要大量内存和时间。按引用传递则涉及传递指向该结构的指针，效率更高。函数还可以直接修改所指向的结构，而无需返回另一个结构。C代码示例eC.34展示了拉伸函数的两个版本，该函数使触点增高2厘米。stretchByReference避免了两次复制大型结构。

C Co示例 eC.34 按值传递结构体或按

名称

```
struct contact stretchByValue(struct contact c) {
    c.height += 0.02;
    return c;
}
void stretchByReference(struct contact *cptr) {
    cptr->height += 0.02;
}
int main(void) {
    struct contact George;
    George.height = 1.4; // poor fellow has been stooped over
    George = stretchByValue(George); // stretch for the stars
    stretchByReference(&George);    // and stretch some more
}
```

C.8.6 typedef

C 还允许您使用 typedef 语句为数据类型定义自己的名称。例如，当 struct contact 被频繁使用时，书写起来会变得繁琐，因此我们可以定义一个新类型 contact，并像 C 代码示例 eC.35 中所示那样使用它。

C代码示例 eC.35 使用typedef创建自定义类型

```
typedef struct contact { char name[30]; int phone; float height; // 以米为单位 } contact;
// 将contact定义为"struct contact"的简写 contact c1; // 现在我们可以将变量声明为contact类型
```

typedef 可用于创建与基本类型占用相同内存量的新类型。C 代码示例 eC.36 将 byte 和 bool 定义为 8 位类型。byte 类型可以更清晰地表明 pos 的目的是作为一个 8 位数字，而非 ASCII 字符。bool 类型则表示该 8 位数字代表 TRUE 或 FALSE。与到处使用 char 相比，这些类型使程序更易于阅读。

C代码示例 eC.36 typedef byte AND bool

```
typedef unsigned char byte; typedef char bool; #define TRUE 1 #define FALSE 0
byte pos = 0x45; bool loveC = TRUE;
```

C 代码示例 eC.37 演示了使用数组定义一个 3 元素向量和一个 3×3 矩阵类型。

C代码示例 eC.37 typedef 向量 AND 矩阵

```
typedef double vector[3]; typedef double matrix[3][3]; vector a = {4.5, 2.3, 7.0};
matrix b = {{3.3, 4.7, 9.2}, {2.5, 4, 9}, {3.1, 99.2, 88}};
```

C.8.7 动态内存分配*

到目前为止的所有示例中，变量都是声明为*statically*的；也就是说，它们的大小在编译时是已知的。这对于大小可变的数组和字符串来说可能是个问题，因为数组必须声明得足够大，以容纳程序可能遇到的最大尺寸。另一种方法是*dynamically*在运行时当实际大小已知时分配内存。

来自 `stdlib.h` 的 `malloc` 函数分配指定大小的内存块并返回指向它的指针。如果没有足够的内存可用，则返回 `NULL` 指针。例如，以下代码分配十个 16 位整数 ($10 \times 2 = 20$ 字节)。 `sizeof` 运算符返回类型或变量的大小（以字节为单位）。

```
// 动态分配20字节的内存int16_t *data = malloc(10 * sizeof(int16_t));
```

C 代码示例 eC.38 展示了动态分配与释放。该程序接受可变数量的输入，将其存储在动态分配的数组中，并计算它们的平均值。所需的内存量取决于数组中的元素数量以及每个元素的大小。例如，如果 `int` 是 4 字节的变量，并且需要十个元素，则会动态分配 40 字节。`free` 函数释放内存，以便后续可用于其他用途。未能释放动态分配的内存称为 *memory leak*，应予以避免。

C Code Example eC.38 DYNAMIC MEMORY ALLOCATION AND DEALLOCATION

```
// Dynamically allocate and deallocate an array using malloc and free
#include <stdlib.h>

// Insert getMean function from C Code Example eC.24.

int main(void) {
    int len, i;
    int *nums;

    printf("How many numbers would you like to enter? ");
    scanf("%d", &len);
    nums = malloc(len*sizeof(int));
    if (nums == NULL) printf("ERROR: out of memory.\n");
    else {
        for (i = 0; i < len; i++) {
            printf("Enter number: ");
            scanf("%d", &nums[i]);
        }
        printf("The average is %f\n", getMean(nums, len));
    }
    free(nums);
}
```


C.8.8 链表*

*linked list*是一种常见的数据结构，用于存储数量可变的元素。列表中的每个元素都是一个结构体，包含一个或多个数据字段以及指向下一个元素的链接。列表中的第一个元素称为*head*。链表展示了结构体、指针和动态内存分配中的许多概念。

C代码示例eC.39描述了一个用于存储计算机用户账户的链表，以容纳可变数量的用户。每个用户拥有用户名、密码、唯一的用户识别号（UID）以及一个指示其是否具有管理员权限的字段。列表中的每个元素均为userL类型，包含该用户的所有信息以及指向列表中下一个元素的链接。指向列表头部的指针存储在一个名为users的全局变量中，初始设置为NULL以表示没有用户。

该程序定义了插入、删除、查找用户以及统计用户数量的函数。insertUser函数为新列表元素分配空间，并将其添加到列表头部。deleteUser函数遍历列表，直到找到指定的UID，然后移除该元素，调整前一个元素的链接以跳过被删除的元素，并释放被删除元素占用的内存。findUser函数遍历列表，直到找到指定的UID，并返回指向该元素的指针，如果未找到UID则返回NULL。numUsers函数统计列表中的元素数量。

C代码示例 eC.39 链表

```
#include <stdlib.h>
#include <string.h>

typedef struct userL {
    char uname[80];      // user name
    char passwd[80];     // password
    int uid;             // user identification number
    int admin;           // 1 indicates administrator privileges
    struct userL *next;
} userL;

userL *users = NULL;

void insertUser(char *uname, char *passwd, int uid, int admin) {
    userL *newUser;

    newUser = malloc(sizeof(userL)); // create space for new user
    strcpy(newUser->uname, uname);   // copy values into user fields
    strcpy(newUser->passwd, passwd);
    newUser->uid = uid;
    newUser->admin = admin;
    newUser->next = users;           // insert at start of linked list
    users = newUser;
}
```

```
void deleteUser(int uid) { // delete first user with given uid
    userL *cur = users;
    userL *prev = NULL;

    while (cur != NULL) {
        if (cur->uid == uid) { // found the user to delete
            if (prev == NULL) users = cur->next;
            else prev->next = cur->next;
            free(cur);
            return; // done
        }
        prev = cur; // otherwise, keep scanning through list
        cur = cur->next;
    }
}

userL *findUser(int uid) {
    userL *cur = users;
    while (cur != NULL) {
        if (cur->uid == uid) return cur;
        else cur = cur->next;
    }
    return NULL;
}

int numUsers(void) {
    userL *cur = users;
    int count = 0;
    while (cur != NULL) {
        count++;
        cur = cur->next;
    }
    return count;
}
```

摘要

指针：指针存储变量的地址。► 数组：数组是使用方括号 [] 声明的相似元素列表。► 字符：char 类型可以存储小整数或用于表示文本或符号的特殊代码。► 字符串：字符串是以空终止符 0x00 结尾的字符数组。► 结构体：结构体存储一组相关变量。► 动态内存分配：malloc 是标准库 (std lib.h) 中的内置函数，用于在程序运行时分配内存。free 在使用后释放内存。► 链表：链表是一种常见的数据结构，用于存储数量可变的元素。

C.9 标准库

程序员通常使用各种标准函数，例如打印和三角函数操作。为了让每个程序员不必从头编写这些函数，C语言提供了*libraries*个常用函数库。每个库都有一个头文件和一个关联的目标文件，目标文件是部分编译的C文件。头文件包含变量声明、定义的类型和函数原型。目标文件包含函数本身，并在编译时链接以生成可执行文件。由于库函数调用已编译成目标文件，因此编译时间得以减少。表eC.5列出了一些最常用的C语言库，下文将对每个库进行简要说明。

C . 9 . 1 stdio

标准输入/输出库 `stdio.h` 包含用于控制台打印、读取键盘输入以及读写文件的命令。要使用这些函数，必须在 C 文件顶部包含该库：

```
#include <stdio.h>
```

表 eC.5 常用C语言库

C Library Header File	Description
<code>stdio.h</code>	Standard input/output library. Includes functions for printing or reading to/from the screen or a file (<code>printf</code> , <code>fprintf</code> and <code>scanf</code> , <code>fscanf</code>) and to open and close files (<code>fopen</code> and <code>fclose</code>).
<code>stdlib.h</code>	Standard library. Includes functions for random number generation (<code>rand</code> and <code>srand</code>), for dynamically allocating or freeing memory (<code>malloc</code> and <code>free</code>), terminating the program early (<code>exit</code>), and for conversion between strings and numbers (<code>atoi</code> , <code>atol</code> , and <code>atof</code>).
<code>math.h</code>	Math library. Includes standard math functions such as <code>sin</code> , <code>cos</code> , <code>asin</code> , <code>acos</code> , <code>sqrt</code> , <code>log</code> , <code>log10</code> , <code>exp</code> , <code>floor</code> , and <code>ceil</code> .
<code>string.h</code>	String library. Includes functions to compare, copy, concatenate, and determine the length of strings.

printf
print formatted 函数 printf 用于在控制台显示文本。其必需的输入参数是一个用引号 "" 括起来的字符串。该字符串包含文本以及用于打印变量的可选命令。要打印的变量列在字符串之后，并使用表 eC.6 中所示的格式代码进行打印。C 代码示例 eC.40 给出了一个简单的 printf 示例。

表 eC.6 用于打印变量的 printf 格式代码

Code	Format
%d	Decimal
%u	Unsigned decimal
%x	Hexadecimal
%o	Octal
%f	Floating-point number (float or double)
%e	Floating-point number (float or double) in scientific notation (e.g., 1.56e7)
%c	Character (char)
%s	String (null-terminated array of characters)

C代码示例 eC.40 使用printf打印到控制台

```
// 简单的打印函数#include <stdio.h>int num = 42;int main(void) { printf("答案是 %d\n", num);}
```

控制台输出：
答案是42。

浮点格式 (float和double) 默认打印小数点后六位。要更改精度，请将%f替换为%w.df，其中w是数字的最小宽度，d是要打印的小数位数。请注意，小数点计入宽度计数。在C代码示例eC.41中，π以总

共四个字符，其中两位在小数点后：3.14。e 以总共八个字符打印，其中三位在小数点后。由于小数点前只有一位数字，因此前面填充了三个空格以达到要求的宽度。c 应以五个字符打印，其中三位在小数点后。但它太宽无法容纳，因此要求的宽度被覆盖，同时保留小数点后的三位数字

。

C代码示例 eC.41 用于打印的浮点数格式

```
// 以不同格式打印浮点数float pi = 3.14159, e = 2.7182, c = 2.998e8;printf("pi = %4.2f\ne = %8.3f\nc = %5.3f\n", pi, e, c);
```

控制台输出：

```
π = 3.14 e = 2.718 c
= 299800000.000
```

由于 % 和 \ 在打印格式化中被使用，要打印这些字符本身，必须使用C代码示例eC.42中所示的特殊字符序列。

C代码示例 eC.42 使用printf打印%和\

```
// 如何向控制台打印 % 和 \ printf("这里是一些特殊字符：%% \\ \n");
```

控制台输出：

```
以下是一些特殊字符：% \
```

scanf

scanf 函数读取键盘输入的文本。它使用与 printf 相同的格式代码。C 代码示例 eC.43 展示了如何使用 scanf。当遇到 scanf 函数时，程序会等待用户输入一个值后才继续执行。scanf 的参数是一个指示一个或多个格式代码的字符串，以及指向应存储结果的变量的指针。

**C Code Example eC.43 READING USER INPUT FROM THE KEYBOARD
WITH scanf**

```
// Read variables from the command line
#include <stdio.h>

int main(void)
{
    int a;
    char str[80];
    float f;

    printf("Enter an integer.\n");
    scanf("%d", &a);
    printf("Enter a floating-point number.\n");
    scanf("%f", &f);
    printf("Enter a string.\n");
    scanf("%s", str); // note no & needed: str is a pointer
}
```

文件操作

许多程序需要读写文件，无论是操作已存储在文件中的数据，还是记录大量信息。在C语言中，必须先使用`fopen`函数打开文件。然后，可以通过`fscanf`或`fprintf`以类似于控制台读写的方式进行读取或写入。最后，应使用`fclose`命令关闭文件。

`fopen`函数接受文件名和`print mode`作为参数。它返回一个类型为`FILE*`的`file pointer`。如果`fopen`无法打开文件，则返回`NULL`。当尝试读取不存在的文件或写入已被其他程序打开的文件时，可能会发生这种情况。模式如下：

"w"：写入文件。如果文件存在，则会被覆盖。

"r"：从文件读取。

"a"：追加到现有文件的末尾。如果文件不存在，则创建该文件。

C代码示例eC.44展示了如何打开、写入和关闭文件。始终检查文件是否成功打开，并在未成功时提供错误信息是一种良好的编程习惯。`exit`函数将在C.9.2节中讨论。`fprintf`函数与`printf`类似，但它还将文件指针作为输入参数，以指定写入哪个文件。`fclose`关闭文件，确保所有信息实际写入磁盘，并释放文件系统资源。

C代码示例 eC.44 使用fprintf打印到文件

```
// Write "Testing file write." to result.txt
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    FILE *fptr;

    if((fptr = fopen("result.txt", "w")) == NULL) {
        printf("Unable to open result.txt for writing.\n");
        exit(1); // exit the program indicating unsuccessful execution
    }
    fprintf(fptr, "Testing file write.\n");
    fclose(fptr);
}
```

在C代码示例eC.44中，惯用的做法是在一行代码中打开文件并检查文件指针是否为NULL。然而，你也可以轻松地将该功能拆分为两行：`fptr = fopen("result.txt", "w");` if (`fptr == NULL`) ...

C代码示例eC.45演示了使用fscanf从名为data.txt的文件中读取数字。首先必须打开文件以供读取。程序随后使用feof函数检查是否已到达文件末尾。只要程序未到达文件末尾，它就会读取下一个数字并将其打印到屏幕上。同样，程序在最后关闭文件以释放资源。

C代码示例 eC.45 使用fscanf从文件读取输入

```
#include <stdio.h>
int main(void) {
    FILE *fptr;
    int data;

    // read in data from input file
    if ((fptr = fopen("data.txt", "r")) == NULL) {
        printf("Unable to read data.txt\n");
        exit(1);
    }
    while (!feof(fptr)) { // check that the end of the file hasn't been reached
        fscanf(fptr, "%d", &data);
        printf("Read data: %d\n", data);
    }
    fclose(fptr);
}
```

data.txt

25 32 14 89

Console Output:

Read data: 25
Read data: 32
Read data: 14
Read data: 89

其他便捷的 `stdio` 函数

`sprintf` 函数将字符打印到字符串中，而 `sscanf` 从字符串中读取变量。`fgetc` 函数从文件中读取单个字符，而 `fgets` 则将一整行读取到字符串中。

`fscanf` 在读取和解析复杂文件方面的能力相当有限，因此通常更简单的方法是使用 `fgets` 逐行读取，然后通过 `sscanf` 或使用 `fgetc` 逐个检查字符的循环来处理该行。

C.9.2 `stdlib`

标准库 `stdlib.h` 提供了通用函数，包括随机数生成（`rand` 和 `srand`）、动态内存分配（`malloc` 和 `free`，已在 C.8.7 节讨论）、提前退出程序（`exit`）以及数字格式转换。要使用这些函数，请在 C 文件顶部添加以下代码行。

```
#include <stdlib.h>
```

`rand` 和 `srand`

`rand` 返回一个伪随机整数。伪随机数具有随机数的统计特性，但遵循从称为 *seed* 的初始值开始的确定性模式。要将数字转换为特定范围，请使用取模运算符（`%`），如 C 代码示例 eC.46 中所示，范围为 0 到 9。`x` 和 `y` 的值将是随机的，但每次运行此程序时它们都相同。示例控制台输出如下所示。

C代码示例 eC.46 使用`rand`生成随机数

```
#include <stdlib.h> int x, y; x = rand(); // x = 一个随机整数 y
= rand() % 10; // y = 一个0到9之间的随机数 printf("x = %d, y = %
d\n", x, y);
```

控制台输出：

```
x = 1481765933, y = 3
```

程序员通过更改种子，使程序每次运行时生成不同的随机数序列。这是通过调用 `srand` 函数实现的，该函数将种子作为输入参数。如

C代码示例eC.47中，种子本身必须是随机的，因此典型的C程序通过调用time函数来赋值，该函数返回当前时间的秒数。

C Code Example eC.47 SEEDING THE RANDOM NUMBER GENERATOR
USING srand

```
// Produce a different random number each run
#include <stdlib.h>
#include <time.h>           // needed to call time()

int main(void) {
    int x;
    srand(time(NULL));      // seed the random number generator
    x = rand() % 10;         // random number from 0 to 9
    printf("x = %d\n", x);
}
```

退出
exit 函数会提前终止程序。它接受一个参数，该参数返回给操作系统以指示终止原因。0 表示正常完成，而非零值则表示错误状态。

格式转换：atoi、atol、atof
标准库提供了使用atoi和atof将ASCII字符串转换为整数或双精度浮点数的函数，如C代码示例eC.48所示。这在从文件读取混合数据（字符串和数字的混合）或处理数值型命令行参数时特别有用，如第C.10.3节所述。

C代码示例 eC.48 格式转换

```
// 将ASCII字符串转换为整数和浮点数 #include <stdlib.h>
> int main(void) { int x; double z; x = atoi("42"); z
= atof("3.822"); printf("x = %d\tz = %f\n", x, z);

}

控制台输出：
x = 42 z = 3.822000
```

由于历史原因，time函数通常返回自1970年1月1日00:00 UTC以来的当前时间（以秒为单位）。UTC代表协调世界时，与格林威治标准时间（GMT）相同。这个日期恰好在1969年贝尔实验室的一个团队（包括丹尼斯·里奇和布莱恩·克尼汉）创建UNIX操作系统之后。类似于除夕派对，一些UNIX爱好者会举办庆祝活动，以纪念time返回的特定数值。例如，在2009年2月1日23:31:30 UTC，time返回了1,234,567,890。到2038年，32位UNIX时钟将溢出，回退到1901年。

C.9.3 数学

数学库math.h提供了常用的数学函数，如三角函数、平方根和对数。C代码示例eC.49展示了如何使用其中一些函数。要使用数学函数，请在C文件中添加以下代码行：

```
#include <math.h>
```

C代码示例 eC.49 数学函数

```
// Example math functions
#include <stdio.h>
#include <math.h>

int main(void) {
    float a, b, c, d, e, f, g, h;

    a = cos(0);           // a = 1, note: the input argument is in radians
    b = 2 * acos(0);       // b = pi (acos means arc cosine)
    c = sqrt(144);         // c = 12
    d = exp(2);            // d = e^2 = 7.389056
    e = log(7.389056);     // e = 2 (natural logarithm, base e)
    f = log10(1000);       // f = 3 (log base 10)
    g = floor(178.567);    // g = 178, rounds to next lowest whole number
    h = pow(2, 10);        // h = 2^10 (i.e., 2 raised to the 10th power)

    printf("a = %.0f, b = %f, c = %.0f, d = %.0f, e = %.2f, f = %.0f, g = %.2f, h = %.2f\n",
           a, b, c, d, e, f, g, h);
}
```

控制台输出：

```
a = 1, b = 3.141593, c = 12, d = 7, e = 2.00, f = 3, g = 178.00, h = 1024.00
```

C.9.4 字符串

字符串库 string.h 提供了常用的字符串操作函数。关键函数包括：

```
// 将 src 复制到 dst 并返回 dst char *strcpy(char
*dst, char *src);
```

```
// 将 src 拼接（追加）到 dst 的末尾并返回 dst char *strcat(char *dst, char *src);
```

```
// 比较两个字符串。相等返回0，否则返回非零值 int strcmp(char *s1, char *s2)
;
```

```
// 返回字符串str的长度，不包括空终止符int strlen(char *str);
```

C.10 编译器和命令行选项

尽管我们介绍了相对简单的C程序，但实际程序可能由数十甚至数千个C文件组成，以实现模块化、可读性和多程序员协作。本节将介绍如何编译分布在多个C文件中的程序，并展示如何使用编译器选项和命令行参数。

C.10.1 编译多个C源文件

多个C文件通过将所有文件名列在编译行上来编译成单个可执行文件，如下所示。请记住，这组C文件仍然只能包含一个main函数，通常放在名为main.c的文件中。

```
gcc main.c file2.c file3.c
```

C.10.2 编译器选项

编译器选项允许程序员指定输出文件名和格式、优化等内容。编译器选项并未标准化，但表eC.7列出了常用的选项。每个选项通常在命令行中以短横线 (-) 开头，如下所示。例如，“-o”选项允许程序员指定除默认的a.out之外的输出文件名。存在大量选项；可以通过在命令行中输入gcc --help来查看它们。

表 eC.7 编译器选项

Compiler Option	Description	Example
-o outfile	Specifies output file name	gcc -o hello hello.c
-S	Create assembly language output file (not executable)	gcc -S hello.c this produces hello.s
-v	Verbose mode—prints the compiler results and processes as compilation completes	gcc -v hello.c
-Olevel	Specify the optimization level (level is typically 0 through 3), producing faster and/or smaller code at the expense of longer compile time	gcc -O3 hello.c
--version	List the version of the compiler	gcc --version
--help	List all command line options	gcc --help
-Wall	Print all warnings	gcc -Wall hello.c

C.10.3 命令行参数

与其他函数类似，main函数也可以接收输入变量。但与其他函数不同的是，这些参数是在命令行中指定的。如C代码示例eC.50所示，argc代表 *argument count*，表示命令行中参数的数量。argv代表 *argument vector*，是一个包含命令行中字符串的数组。例如，假设C代码示例eC.50中的程序被编译成名为testargs的可执行文件。当在命令行中输入以下行时，argc的值为4，数组argv的值为{"/.testargs", "arg1", "25", "lastarg!"}。请注意，可执行文件名被计为第0th个（即最左侧）参数。输入此命令后的控制台输出如下所示，位于C代码示例eC.50下方。

```
gcc -o testargs testargs.c ./testargs arg  
1 25 lastarg!
```

需要数值参数的程序可以使用stdlib.h中的函数将字符串参数转换为数字。

C代码示例 eC.50 命令行参数

```
// 打印命令行参数#include <stdio.h>int main(int argc, char *argv[]) { int i; for (i = 0; i < argc; i++) printf("arg  
v[%d] = %s\n", i, argv[i]);}
```

控制台输出

```
argv[0] = ./testargs argv[1]  
= arg1 argv[2] = 25 argv[3]  
= lastarg!
```

C.11 常见错误

与任何编程语言一样，在编写非平凡的C程序时，几乎肯定会出错。以下是一些C语言编程中常见错误的描述。其中一些错误尤其令人困扰，因为它们能够编译通过，但运行结果却与程序员的预期不符。

C代码错误 eC.1 scanf中缺少&

Erroneous Code	Corrected Code
<pre>int a; printf("Enter an integer:\t"); scanf("%d", a); // missing & before a</pre>	<pre>int a; printf("Enter an integer:\t"); scanf("%d", &a);</pre>

C Code Mistake eC.2 USING = INSTEAD OF == FOR COMPARISON

Erroneous Code	Corrected Code
<pre>if (x = 1) // always evaluates as TRUE printf("Found!\n");</pre>	<pre>if (x == 1) printf("Found!\n");</pre>

C Code Mistake eC.3 INDEXING PAST LAST ELEMENT OF ARRAY

Erroneous Code	Corrected Code
<pre>int array[10]; array[10] = 42; // index is 0-9</pre>	<pre>int array[10]; array[9] = 42;</pre>

C Code Mistake eC.4 USING = IN #define STATEMENT

Erroneous Code	Corrected Code
<pre>// replaces NUM with "= 4" in code #define NUM = 4</pre>	<pre>#define NUM 4</pre>

C Code Mistake eC.5 USING AN UNINITIALIZED VARIABLE

Erroneous Code	Corrected Code
<pre>int i; if (i == 10) // i is uninitialized ...</pre>	<pre>int i = 10; if (i == 10) ...</pre>

C Code Mistake eC.6 NOT INCLUDING PATH OF USER-CREATED HEADER FILES

Erroneous Code	Corrected Code
<pre>#include "myfile.h"</pre>	<pre>#include "othercode\myfile.h"</pre>

调试技巧是通过实践获得的，但这里有一些提示。

- 从编译器报告的第一个错误开始修复。后续错误可能是该错误的连带影响。修复该错误后，重新编译并重复此过程，直到所有错误（至少是编译器能捕获的那些！）都被修复。
- 当编译器指出某行有效代码存在错误时，请检查该行上方的代码（例如，是否缺少分号或花括号）。
- 必要时，将复杂的语句拆分为多行。
- 使用printf输出中间结果。
- 当结果不符合预期时，从代码偏离预期的位置first开始调试。
- 关注所有编译器警告。虽然某些警告可以忽略，但其他警告可能提醒你注意更隐蔽的代码错误——这些错误能通过编译，但无法按预期运行。

C Code Mistake eC.7 USING LOGICAL OPERATORS (!, ||, &&) INSTEAD OF BITWISE (~, |, &)

Erroneous Code	Corrected Code
<pre>char x = !5; // logical NOT: x = 0 char y = 5 2; // logical OR: y = 1 char z = 5&&2; // logical AND: z = 1</pre>	<pre>char x = ~5; // bitwise NOT: x = 0b11111010 char y = 5 2; // bitwise OR: y = 0b00000111 char z = 5&2; // logical AND: z = 0b00000000</pre>

C代码错误 eC.8 在switch/case语句中忘记break

Erroneous Code	Corrected Code
<pre>char x = 'd'; ... switch (x) { case 'u': direction = 1; case 'd': direction = 2; case 'l': direction = 3; case 'r': direction = 4; default: direction = 0; } // direction = 0</pre>	<pre>char x = 'd'; ... switch (x) { case 'u': direction = 1; break; case 'd': direction = 2; break; case 'l': direction = 3; break; case 'r': direction = 4; break; default: direction = 0; } // direction = 2</pre>

C代码错误 eC.9 缺少花括号 {}

Erroneous Code	Corrected Code
<pre>if (ptr == NULL) // missing curly braces printf("Unable to open file.\n"); exit(1); // always executes</pre>	<pre>if (ptr == NULL) { printf("Unable to open file.\n"); exit(1); }</pre>

C代码错误 eC.10 在函数声明之前使用函数

Erroneous Code	Corrected Code
<pre>int main(void) { test(); } void test(void) { ... }</pre>	<pre>void test(void) { ... } int main(void) { test(); }</pre>

C代码错误 eC.11 声明同名的局部变量和全局变量

Erroneous Code	Corrected Code
<pre>int x = 5; // global declaration of x int test(void) { int x = 3; // local declaration of x ... }</pre>	<pre>int x = 5; // global declaration of x int test(void) { int y = 3; // local variable is y ... }</pre>

C语言错误 eC.12 在声明后尝试使用 {} 初始化数组

错误代码	修正代码
<pre>int scores[3] = {88, 79, 93}; scores = {93, 81, 97}; // won't compile</pre>	

错误代码	修正代码
<pre>int scores[3] = {88, 79, 93}; int scores2[3] = {88, 79, 93}; scores2 = scores;</pre>	<pre>int scores[3] = {88, 79, 93}; int scores2[3]; for (i = 0; i < 3; i++) scores2[i] = scores[i];</pre>

C代码错误 eC.14 忘记在do/while循环后加分号

错误代码	修正代码
<pre>int num; do { num = getNum(); } while (num < 100);</pre>	<pre>int num; do { num = getNum(); } while (num < 100);</pre>

C代码错误 eC.15 在for循环中使用逗号代替分号

错误代码	修正代码
<pre>for (i=0, i < 200, i++) ...</pre>	<pre>for (i = 0; i < 200; i++) ...</pre>

C代码错误 eC.16 整数除法代替浮点除法

Erroneous Code	Corrected Code
<pre>// integer (truncated) division occurs when // both arguments of division are integers float x = 9 / 4; // x = 2.0</pre>	<pre>// at least one of the arguments of // division must be a float to // perform floating-point division float x = 9.0 / 4; // x = 2.25</pre>

C代码错误 eC.17 写入未初始化的指针

Erroneous Code	Corrected Code
<pre>int *y = 77;</pre>	<pre>int x, *y = &x; *y = 77;</pre>

C代码错误 eC.18 巨大的期望（或缺乏期望）REOF

一个常见的初学者错误是编写整个程序（通常缺乏模块化），并期望它第一次就能完美运行。对于非平凡的程序，编写模块化代码并在此过程中逐个测试各个函数至关重要。随着复杂度的增加，调试的难度和时间消耗会呈指数级增长。

另一个常见错误是缺乏预期结果。当这种情况发生时，程序员只能验证代码产生了结果，而无法验证结果是否正确。使用已知输入和预期结果来测试程序，对于验证功能至关重要。

本附录主要讨论了在个人计算机等系统上使用C语言的相关内容。第9章（作为网络补充材料提供）介绍了如何利用C语言对SparkFun公司基于RISC-V微控制器的RED-V RedBoard进行编程，该开发板可用于嵌入式系统。微控制器通常采用C语言编程，因为该语言既能像汇编语言一样对硬件进行近乎底层的控制，又更加简洁且编写速度更快。

延伸阅读

柏林 L., *The Man Behind the Microchip: Robert Noyce and the Invention of Silicon Valley*, 牛津大学出版社, 2005年。

罗伯特·诺伊斯 (Robert Noyce) 的精彩传记, 他是微芯片的发明者之一, 也是仙童半导体和英特尔的创始人。对于任何考虑在硅谷工作的人来说, 这本书深入揭示了该地区的文化, 而这种文化受诺伊斯的影响比任何其他人都更为深远。

Ciletti M., *Advanced Digital Design with the Verilog HDL, 2nd ed.*, Prentice Hall, 2010.

关于Verilog 2005 (而非SystemVerilog) 的一个良好参考。

Colwell R., *The Pentium Chronicles: The People, Passion, and Politics Behind Intel's Landmark Chips*, Wiley, 2005.

一位项目负责人讲述英特尔奔腾芯片几代产品开发的内幕故事。对于考虑进入该领域的人士, 本书提供了对大型设计项目管理的见解, 并揭示了最具商业价值的微处理器系列之一的幕后故事。

Ercegovac M. 和 Lang T., *Digital Arithmetic*, Morgan Kaufmann, 2003年。

最完整的计算机算术系统教材。构建高质量计算机算术单元的绝佳资源。

Hennessy J., 和 Patterson D., *Computer Architecture: A Quantitative Approach, 6th ed.*, Morgan Kaufmann, 2017年。关于高级计算机架构的权威著作。如果你对前沿微处理器的内部工作原理感兴趣, 这本书正适合你。

Kidder T., *The Soul of a New Machine*, Back Bay Books, 1981年。一个关于计算机系统设计的经典故事。三十年后, 这个故事依然引人入胜, 其中关于项目管理和技术的见解仍然发人深省。Patterson D. 和 Waterman A., *The RISC-V Reader: An Open Architecture Atlas*, Strawberry Canyon, 2017年。由两位RISC-V架构师撰写的RISC-V架构简明介绍。

Pedroni V., *Circuit Design and Simulation with VHDL, 2nd ed.*, 麻省理工出版社, 2010年。

展示如何使用VHDL设计电路的参考。SystemVerilog IEEE标准 (IEEE STD 1800)。SystemVerilog硬件描述语言的IEEE标准; 最近更新于2019年。可在ieeexplore.ieee.org获取。VHDL IEEE标准 (IEEE STD 1076)。VHDL的IEEE标准; 最近更新于2017年。可在ieeexplore.ieee.org获取。Wakerly J., *Digital Design: Principles and Practices, 5th ed.*, Pearson, 2018。一本全面且易读的数字设计教材, 也是一本优秀的参考书。Weste N. 和Harris D., *CMOS VLSI Design, 4th ed.*, Addison-Wesley, 2010。超大规模集成 (VLSI) 设计是构建包含大量晶体管的芯片的艺术与科学。这本书由我们最喜爱的作者之一合著, 涵盖了从入门到商业产品中最先进技术的整个领域。

索引

Note: Page numbers followed by “*f*,” “*t*,” and “*b*” refer to figures, tables, and boxes, respectively; page numbers preceded by “*e*” refer to online material.

0、5–6、20。 *See also* 低电平，假 1、5–6、20。 *See also* 高电平，真 32 位数据通路，394 32 位指令，332 64 位架构，373 74xx 系列逻辑，543.e1–2 部件 2: 1 多路选择器 (74157)，543.e4f 3:8 译码器 (74138)，543.e4f 4:1 多路选择器 (74153)，543.e4f 与门 (7408)，543.e3f 三输入与门 (7411)，543.e3f 四输入与门 (7421)，543.e3f 计数器 (74161、74163)，543.e4f 触发器 (7474)，543.e3f 与非门 (7400)，543.e3f 或非门 (7402)，543.e3f 非门 (7404)，543.e3f 或门 (7432)，543.e3f 寄存器 (74377)，543.e4f 三态缓冲器 (74244)，543.e4f 异或门 (7486)，543.e3f #define，545.e5–6 #include，545.e6–7。 *See also* 标准

抽象，2–3 数字。 *See* 数字抽象 累加器，372 低电平有效，72–73 A/D 转换，542.e 37–39 *Ad hoc* 测试，464 ADC。 *See* 模数转换器 (ADC) 加法指令，301, 444, 472–473 加法器，237–244 进位传播，238 超前进位，239–241 全加器，238 半加器，238 HDL 描述，181–182, 198*b*, 461*b* 前缀加法器，241–243 行波进位加法器，238–239 加法，12*f*, 13*b*, 16*b*, 237–244, 301*b*。 *See also* 加法器 二进制，12–13 浮点数，261 有符号二进制，13–17 地址。 *See also* 存储器 物理，521, 521*f* 转换，523–524 虚拟，521。 *See also* 虚拟 存储器 寻址模式，RISC-V，341–342 基址，341 立即数，341 PC 相对，341–342 寄存器，341 超威半导体 (AMD)，300

高级微架构，468–482 分支预测。 *See* 分支预测深流水线。 *See* 深流水线异构多处理器。 *See* 异构多处理器同构多处理器。 *See* 同构多处理器微操作。 *See* 微操作多处理器。 *See* 多处理器多线程。 *See* 多线程乱序处理器。 *See* 乱序处理器寄存器重命名。 *See* 寄存器重命名单指令多数据。 *See* 单指令多数据 (SIMD) 超标量处理器。 *See* 超标量处理器 AHB。 *See* 高级高性能总线 (AHB) AHB-Lite 总线，485 Altera FPGA，278 ALU。 *See* 算术/逻辑单元 (ALU) ALU 解码器，408–409, 409*t* ALUControl，401–402, 457–458 *ALUOp*，408 *ALUResult*，418 *ALUSrc*，404 AMAT。 *See* 平均内存访问时间 (AMAT) AMBA。 *See* 高级微控制器总线架构 (AMBA) AMD。 *See* 超威半导体 (AMD)

库

A

ABI。 *See* 应用程序二进制接口 (ABI)

AMD64, 373 阿姆达尔, 吉恩, 504 阿姆达尔定律, 505 美国信息交换标准代码 (ASCII), 545.e8–9, 545.e27–28 模拟I/O, 542.e31–39 A/D转换, 542.e37–39 D/A转换, 542.e31–34 脉宽调制 (PWM), 542.e34–37 模数转换器 (ADCs), 542.e31, 542.e37–39 分析机, 5 与指令, 309 与门, 18–19, 177*b* 芯片 (7408, 7411, 7421), 543.e3*f* 真值表, 18 *f*, 20*f* 使用CMOS晶体管, 31 与或 (AO) 门, 45 阳极, 25–26 反依赖, 475 专用集成电路 (ASICs), 543.e9 架构状态, 344, 369 对于RISC-V, 393–394 架构, 299 汇编语言, 300–308 指令, 301–302 操作数, 302–308 编译、汇编和加载, 344–355 汇编, 350–353 编译, 348–350 链接, 353–355 加载, 355 内存映射, 344–346 RISC-V架构的演进, 363–366 RISC-V与ARM架构的比较, 365–366 RISC-V与MIPS架构的比较, 365 RISC-V基础指令集及扩展, 364–365 机器语言, 332–344 寻址模式, 341–342 立即数编码, 340–341 I型指令格式, 334–336 解释, 342–343 R型指令格式, 332–334 S/B型指令格式, 336–338

存储程序, 343–344 U/I型指令格式, 338–340 杂项355–363 压缩指令, 362–363 字节序355–356 异常356–360 浮点指令361–362 有符号与无符号指令, 360–361 编程308–332 数组, 317–320 分支311–313 条件语句313–315 函数调用320–330 循环入门, 315–317 逻辑与算术指令308–311 内存, 306–308 伪指令330–332 x86架构, 366–374 宏观视角373–374 指令编码, 371–372 指令369 操作数367–368 特性372–373 寄存器366–367 状态标志, 369 参数320, 545.e24*lso* 按引用传递, 545.e22 按值传递545.e22 算术C运算符, 545.e11–15 电路237–256 HDL运算符, 183*b* RISC-V指令, 309 算术/逻辑单元 (ALU), 247–251 实现, 247–248 在处理器中401–402, 409*b* 数组317–319, 545.e23–27 访问316*b*, 545.e23 字节与字符317–320 比较或赋值, 545.e27–28 声明319*b*, 545.e23 索引317, 545.e23–27 初始化545.e23*b*–e24*b*, 545.e24*b*, 545.e25 作为输入参数, 545.e24*b*–25*b*, 545.e24 多维545.e25–26 ASCII。See 美国信息交换标准代码 (ASCII)

ASIC。See 专用集成电路 (ASIC) 汇编器, 344, 350–351 汇编器指令, 346–348 汇编过程, 344–355 汇编语言, RISC-V, 300–332 指令, 301–302 操作数, 302–308 将高级代码翻译为, 303–332 将机器语言翻译为, 342*b*–343*b* 汇编语言, x86。See x86 布尔代数中的结合律, 60, 61*t* 缓存中的结合律, 505–511 非稳态电路, 117*b* 非对称多处理器。See 异构多处理器 异步电路, 120–121 异步可复位触发器 定义, 114 HDL, 193*b* 异步串行链路, 542.e24。See 通用异步收发器 (UART) 平均内存访问时间 (AMAT), 503–505

B

B型指令格式。See S/B型指令格式 Babbage, Charles, 5 基址寻址, 341 波特率, 542.e24–25 BCD。See 二进制编码十进制 (BCD) 行为建模, 188 基准测试, 397 beq指令。See 大于或等于则分支 (bge) bge指令。See 大于或等于则分支 (bge) 偏移指数, 258–259 大端内存, 355–356, 356*f* HDL中的大端序, 176 二进制加法, 12–13。See also 加法器; 加法 二进制编码十进制 (BCD), 260*b* 二进制编码, 127–128 用于除以3计数器, 129*t* 用于交通灯有限状态机, 133

有符号二进制数, 13–17 无符号, 7–9 二进制到十进制转换, 8*b*, 9*f* 二进制到十六进制转换, 10*b* 双极型晶体管, 24 双极电机驱动, 542.e61*f* 双极信号传输, 542.e25 双极步进电机, 542.e59*f*, 542.e60–63 AIRPAX LB82773-M1, 542.e62*f* 直接驱动电流, 542.e62*f* 双稳态元件, 107–108 位, 6 脏位, 519 最低有效位, 11, 12*f* 最高有效位, 11, 12*f* 符号位, 13–14 使用, 515–516 有效位, 509 位单元, 267 DRAM, 268–269 ROM, 268 SRAM, 269 位混洗, 186 位线, 267 位运算符, 175–176 块, 505 块偏移, 513–514 块大小 (*b*), 505–506 阻塞与非阻塞赋值, 197, 203–207 *blt* 指令。See 小于则分支 (*blt*) BlueSMiRF 银模块, 542.e53–54 蓝牙无线通信, 542.e53–54 BlueSMiRF 银模块, 542.e53–54 类别, 542.e53*t* *bne* 指令。See 不等于则分支 (*bne*) 乔治·布尔, 6 布尔代数, 58–64 公理, 59 方程简化, 63–64 定理, 59–62 布尔方程, 56–58 乘积之和 (POS) 形式, 58 和之乘积 (SOP) 形式, 56–58 布尔逻辑, 6。See also 布尔代数; 逻辑门 布尔定理, 59–62 结合律, 60 合并律, 60

交换律, 60 补集, 60 一致性, 60 覆盖, 60 德摩根定律, 61 分配律, 60 幂等律, 60 恒等律, 59 对合律, 60 零元素, 59–60 相等则分支 (*beq*), 311 大于或等于则分支 (*bge*), 311, 316*b* 小于则分支 (*blt*), 360 不相等则分支 (*bne*), 311–312 分支指令, 311 分支误预测惩罚, 451, 470 分支预测, 470–472 分支目标地址 (BTA), 337, 430–431, 432*f* 分支目标缓冲器, 470–471 分支, 311–313 条件分支, 311–312 无条件分支, see 跳转 面包板, 543.e20 BT A。See 分支目标地址 (BTA) 气泡, 18, 61–62 推送, 61–62, 67–68 气泡, 在流水线中, 448 缓冲器, 18 缺乏, 11 5 三态, 72, 73 错误, 173–174 在C代码中, 545.e44–48 总线, 54 三态, 73*f* 旁路, 446。See also 转发 字节, 11–12, 317–320。See also 字符 最低有效, 11 最高有效, 11 字节偏移, 508 字节可寻址内存, 306–307 大端序, 355 小端序, 355

条件语句。See 条件语句是控制流语句。See 控制

流程语句数据类型。See 数据类型执行程序, 545.e3 函数调用。See 函数调用循环。See 循环运算符。See 运算符简单程序, 545.e3 标准库。See 标准库 变量。See C语言中的变量

Caches, 505–520 地址字段 块偏移量, 513 字节偏移量, 508 设置位, 508 标签, 508 高级设计, 516–520 多级, 516–518 组织方式, 515–516 直接映射, 507–511 全相联, 512 多路组相联, 511–512 参数 块大小 (*b*), 513–514 容量 (*C*), 505 相联度 (*N*), 505 块数 (*B*), 505, 512–514 组数 (*S*), 505 性能 容量, 518 强制, 518 冲突, 512, 518 命中, 502 命中率, 503–505 未命中, 502, 518 未命中率, 503–505 未命中率 *vs.* 缓存参数, 518–519 惩罚, 513 减少, 518–519 替换策略, 527–529 状态位 脏位 (*D*), 528 使用位 (*U*), 528–529 有效位 (*V*), 525 写策略, 519–520 写回, 519, 519*b* 写直达, 519, 519*b*

C

C语言编程, 545.e1–48 常见错误。
See C语言常见错误

CAD。See 计算机辅助设计 (CAD) 被调用者, 320 被调用者保存规则, 326 被调用者保存寄存器, 325 调用者保存规则, 326 调用者保存寄存器, 325 规范形式。See 积之和 (SOP) 形式; 和之积 (POS) 形式 电容器, 26 容量, 缓存容量, 505 容量缺失, 518 进位传播加法器 (CPA)。See 超前进位加法器 (CLA); 前缀加法器; 行波进位加法器 超前进位加法器 (CLA), 239–241, 240*f* case语句, 在HDL中, 199–200。See Switch/case语句 casez, case?, 在HDL中, 203*b* 阴极, 25–26 阴极射线管 (CRT), 542.e47–48。See also VGA (视频图形阵列) 显示器 水平消隐间隔, 542.e47–48 垂直消隐间隔, 542.e47–48 字符型LCD, 542.e44–47 字符 (char), 317–319, 542.e44–47 数组。See also 字符串 C类型, 545.e27 芯片, 26 多处理器, 479–482 斩波恒流驱动, 542.e62 电路 74xx系列。See 74xx系列逻辑专用集成电路 (ASIC), 543.e9 非稳态, 117*b* 异步, 120–121 组合。See 组合逻辑 定义, 53–54 延迟, 86–90 毛刺, 90–93 多输出, 66*b* 优先级, 66 时序。See 时序逻辑 同步, 120–121 同步时序, 121 综合, 174*f*, 176*f*, 177*f* 时序, 86–93, 139–155

CISC。See 复杂指令集计算机 CLA。See 超前进位加法器 (CLA) CLB。See 可配置逻辑块 (CLB) 每指令时钟周期数 (CPI)。See 每指令周期数 (CPI) 时钟周期, 140, 398 时钟偏移, 146–149, 468–469 集群多处理器, 481–482 CMOS。See 互补金属氧化物半导体门 (CMOS) 组合构成, 54–55 组合逻辑, 172*b* 设计, 53–56 布尔代数, 58–64 布尔方程, 56–58 构建模块, 81–86 延迟, 86–90 无关项, 79–80 卡诺图 (K图), 73–81 多级, 67–71 优先级, 56 时序, 86–93 两级, 67–68 X (竞争)。See 竞争 (X) X (无关项)。See 无关项 (X) Z (高阻)。See 高阻 (Z) HDL。See 硬件描述语言 (HDL) 组合定理, 60 命令行参数, 545.e44 RISC-V汇编中的注释, 302 C语言中的注释, 302, 545.e5 SystemVerilog中的注释, 178 V HDL中的注释, 178 C语言中的常见错误, 545.e44–48 比较器, 245–246 硬件中的比较。See 算术逻辑单元 (ALU); 比较器 处理器性能, 397–398 使用ALU, 249 编译器, C语言中, 348–350, 545.e4, 545.e43 互补金属氧化物半导体门 (CMOS), 23–32

补集定理, 60 复杂指令集计算机 (CISC) 架构, 302, 366, 469–470 复杂性管理, 2–5 抽象化, 2–3 纪律性, 3–4 层次化, 4 模块化, 4 规律性, 4 压缩指令, 362–363 强制缺失, 518 计算机辅助设计 (CAD), 127, 171, 246 并发信号赋值语句, 177, 182 条件标志, 369 条件助记符, 332 条件赋值, 179–180 条件分支, 311, 312*b* 条件运算符, 179 条件信号赋值, 179 条件语句, 313–315 在RISC-V汇编中 if, 313 if/else, 313–314 switch/case, 314–315 在C语言中, 545.e17–19 if, 545.e17 if/else, 545.e17 switch/case, 545.e18 在HDL中, 199–200 case, 199–200 casez, case?, 203 if, if/else, 200–203 可配置逻辑块 (CLBs), 167, 278。See also 逻辑元件 (LEs) 冲突缺失, 518 一致性定理, 60, 62*b* 在RISC-V汇编中的常量, 302–308。See 立即数 在C语言中, 545.e5–6 污染延迟, 86–90。See also 短路径 竞争 (x), 67*b*, 71–72 上下文切换, 478 连续赋值语句, 177, 191 控制冒险, 446, 450–453 控制信号, 81, 399, 406*f*, 407–410, 422–435